

Lecture 5: CSS Flexbox, Class Attribute, Forms, and External Stylesheets

Introduction

In this lecture, we will cover essential CSS and HTML concepts, including the flex property, class attributes, form elements, and external stylesheets. These concepts are fundamental for designing responsive and well-structured web pages.

1. CSS Flexbox

What is Flexbox?

Flexbox is a CSS layout model that allows items within a container to be arranged efficiently, even when their sizes are dynamic. It is designed for one-dimensional layouts, meaning it aligns items either in a row (horizontally) or in a column (vertically).

The `display: flex;` Property

To enable Flexbox, you need to set the `display` property of a container to `flex`.

```
.container {  
  display: flex;  
}
```

This makes all direct child elements of `.container` become flex items.

Flexbox Properties

1. `flex-direction`

Determines the direction of the flex items.

- `row` (default): Items are placed left to right.
- `row-reverse`: Items are placed right to left.

- column: Items are stacked vertically.
- column-reverse: Items are stacked in reverse order.

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

2. justify-content

Aligns items along the main axis.

- flex-start: Items align to the start.
- flex-end: Items align to the end.
- center: Items are centered.
- space-between: Items are evenly spaced, with the first at the start and the last at the end.
- space-around: Items have equal spacing around them.
- space-evenly: Items have equal spacing between and around them.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

3. align-items

Aligns items along the cross-axis (perpendicular to flex-direction).

- flex-start: Items align at the start.
- flex-end: Items align at the end.
- center: Items are centered.

- stretch: Items stretch to fill the container.
- baseline: Items align according to their text baseline.

Example:

```
.container {  
  display: flex;  
  align-items: center;  
}
```

Examples

Example 1: Simple Flexbox Layout

```
<html>  
<head>  
<style>  
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 300px;  
  border: 1px solid black;  
}  
.box {  
  width: 100px;  
  height: 100px;  
  margin: 10px;  
  background-color: lightblue;  
}  
</style>  
</head>  
<body>  
<div class="container">  
  <div class="box">1</div>
```

```
<div class="box">2</div>
<div class="box">3</div>
</div>
</body>
</html>
```

Result: Three equally spaced boxes, centered both horizontally and vertically.

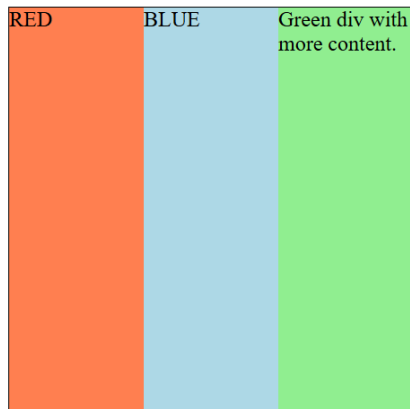


Example 2:

```
<html>
<head>
<style>
#main {
width: 300px;
height: 300px;
border: 1px solid black;
display: flex;
}
#main div {
flex: 1;
}
</style>
</head>
<body>
<h1>The flex Property</h1>
<div id="main">
<div style="background-color:coral;">RED</div>
```

```
<div style="background-color:lightblue;">BLUE</div>
<div style="background-color:lightgreen;">Green div with more
content.</div>
</div>
</body> </html>
```

The flex Property



Example 3: Responsive Navbar with Flexbox

```
<html>
<head>
<style>
.navbar {
  display: flex;
  justify-content: space-between;
  background-color: #333;
  padding: 10px;
}
.navbar a {
  color: white;
  text-decoration: none;
  padding: 10px;
}
</style>
</head>
<body>
<div class="navbar">
```

```
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Contact</a>
</div>
</body>
</html>
```

Result: A responsive navigation bar with evenly spaced links.

2. HTML Class Attribute

The class attribute allows multiple HTML elements to share the same styling by assigning a common name. This is useful for maintaining consistency and reducing redundant code.

Example 5: Using class for Styling

```
<html>
<head>
<style>
.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}
</style>
</head>
<body>
<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
<div class="city">
  <h2>Paris</h2>
```

```
<p>Paris is the capital of France.</p>
</div>
<div class="city">
  <h2>Tokyo</h2>
  <p>Tokyo is the capital of Japan.</p>
</div>
</body>
</html>
```



Key Points:

- The .city class applies the same styles to all <div> elements that have the city class.
- Classes can also be used in JavaScript to manipulate elements dynamically.

3. HTML Forms

Forms are an essential part of web applications as they allow users to input and submit data.

Form Attributes

1. **action:** Specifies the URL where form data should be sent.

2. **method:** Defines how form data is submitted (GET or POST).
3. **target:** Determines where the response is displayed:
 - `_blank`: Opens in a new tab.
 - `_self`: Opens in the same window (default).
 - `_parent`: Opens in the parent frame.
 - `_top`: Opens in the full body of the window.

The HTML `<form>` Elements:

The HTML `<form>` element can contain one or more of the following form elements:

- `<input>`
- `<label>`
- `<select>` `<option>`
- `<textarea>`
- `<button>`
- `<Submit>`
- `<Reset>`

The `<input>` tag specifies an input field where the user can enter data. The `<input>` element is the most important form element. The `<input>` element can be displayed in several ways, depending on the type attribute. The different input types are as follows:

- `<input type="button">`
- `<input type="checkbox">`
- `<input type="color">`
- `<input type="date">`
- `<input type="password">`
- `<input type="radio">`
- `<input type="reset">`
- `<input type="submit">`

The Method Attribute

The method attribute specifies the HTTP method to be used when submitting the form data.

The form-data can be sent as URL variables (with method="get") or as HTTP post transaction (with method="post").

The default HTTP method when submitting form data is GET.

Notes on GET:

- Appends the form data to the URL, in name/value pairs
- NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

Notes on POST:

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked

Example:

```
<form action="submit.php" method="get">
  <input type="text" name="name" value="John">
  <input type="submit">
</form>
```

The URL after submission might be:

<https://example.com/submit.php?name=John>

```
<form action="submit.php" method="post">
  <input type="text" name="name" value="John">
  <input type="submit">
</form>
```

The data is sent in the request body, so the URL remains without any query parameters. <https://example.com/submit.php>

Example 6: Creating a Form

```
<html>
<head> </head>
<body>
<form action="Lecture2.html" method="get">
<label for="user">user name</label>
<Input type="text" name="user" id="user1"
placeholder="Enter your name"><br>
<label>Password</label>
<Input type="Password" name="pass"><br>
<input type="checkbox" id="ch"><label for="ch">Remember
me</label> <br>
<input type="Radio" id="m" name="Gen">
<label for="m">Male</label>
<input type="Radio" id="f" name="Gen">
<label for="m">Female</label><br>
<input type="Radio" id="m1" name="mm">
<label for="m">Single</label>
<input type="Radio" id="f1" name="mm">
<label for="m">Married</label>
<Input type="color">
<input type="date">
<select name="a" id="aa">
<option>Baghdad</option>
<option>London</option>
<option>Paris</option>
</select> <br>
```

```
<Input type="submit">
<Input type="Reset" >
</body>
</html>
```

Result:

user name

Password

☐ Remember me

☐ Male ☐ Female

☐ Single ☐ Married

4. CSS Forms Styling

CSS can be used to improve the appearance of forms.

Example 7: Styling Input Fields

```
input[type=text], select, textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 4px;
    resize: vertical;
}

input[type=submit] {
    background-color: #04AA6D;
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
```

```

    float: right;
}
input[type=submit]:hover {
    background-color: #45a049;
}
.container {
    background-color: #f2f2f2;
    padding: 20px;
    border-radius: 5px;
}

```

Responsive Form Example

```

<div class="container">
  <form action="/action_page1.html">
    <div class="row">
      <label for="fname">First Name:</label>
      <input type="text" id="fname" name="firstname"
placeholder="Your name..">
    </div>
    <div class="row">
      <label for="lname">Last Name:</label>
      <input type="text" id="lname" name="lastname"
placeholder="Your last name..">
    </div>
    <div class="row">
      <label for="country">Country:</label>
      <select id="country" name="country">
        <option value="australia">Australia</option>
        <option value="canada">Canada</option>
        <option value="usa">USA</option>
      </select>
    </div>
    <input type="submit" value="Submit">
  </form>
</div>

```

Result without style

First Name:

Last Name:

Country:

Result with Style

First Name:

Last Name:

Country:

5. External Stylesheets

Instead of writing CSS in the `<style>` tag, we can use an external CSS file for better code organization.

Example 8: `style.css`

```
body {  
    color: blue;  
    background-color: coral;  
    background-image: url("p1.jpg");  
    background-repeat: repeat;  
    background-position: center;  
    background-size: 50%;  
}  
  
p {  
    color: rgb(50,100,150);  
    border: 10px solid brown;  
}  
  
h2 {  
    text-align: right;
```

```
    font-size: 50px;  
}
```

Linking the Stylesheet in HTML:

```
<link rel="stylesheet" href="style.css">
```